



UNIP
UNIVERSIDADE PAULISTA

Roteiros

Prática de Programação Orientada a Objetos

Organização do conjunto

Este documento conduz a construção progressiva de um Sistema de Gestão de Biblioteca. A mesma solução evolui de uma interface Swing para persistência, organização em camadas, módulo web JSP/JSTL e geração de relatórios em planilhas.

As oito atividades usam o domínio didático Sistema de Gestão de Biblioteca. O professor pode substituir o domínio, desde que preserve os contratos, testes e critérios.

Atividade	Eixo
1	Primeira interface visual orientada a objetos com AWT e Swing
2	Layouts, menus, navegação por cartões e tabelas Swing
3	Eventos de ação e de janela com controladores responsivos
4	Conexão MySQL e operações CRUD seguras com JDBC
5	DTO/VO, MVC e Data Access Object aplicados à solução
6	Persistência de entidades e relacionamentos com Hibernate ORM
7	Módulo web com HTML, Servlets, JSP e JSTL
8	Geração de relatórios Excel com Apache POI

Verificação: As atividades são cumulativas. Antes de iniciar uma unidade, mantenha uma versão executável da unidade anterior.

Verificação: Antes de cada atividade, criar uma branch ou tag e confirmar que a versão anterior compila, executa e possui configuração reproduzível.

Objetivo

Criar a janela inicial do sistema, organizar componentes Swing e separar a construção da interface das regras de domínio.

Etapa 1 – Preparar o projeto

1. Criar o projeto Java e registrar a versão do JDK e a ferramenta de construção usadas.
2. Criar os pacotes domain, view, controller e repository, ainda que alguns permaneçam vazios.
3. Modelar inicialmente Livro, Autor e Categoria com atributos e invariantes essenciais.
4. Criar uma classe de inicialização que abra a interface na Event Dispatch Thread.

Verificação: O projeto compila, a estrutura está registrada e a aplicação abre sem erro.

Etapa 2 – Construir a janela principal

5. Criar um JFrame com título, tamanho inicial, política de fechamento e posição definida.
6. Adicionar um painel principal e componentes para título, autor, ISBN e categoria.
7. Definir nomes claros para campos e botões, evitando variáveis genéricas.
8. Aplicar rótulos e mensagens que permitam compreender a ação esperada.

Verificação: A janela apresenta todos os campos sem sobreposição e com textos legíveis.

Etapa 3 – Integrar objetos do domínio

9. Ler os valores dos componentes por um método próprio da view.

10. Validar obrigatoriedade e formato antes de construir o objeto Livro.

11. Exibir mensagens de sucesso ou falha com JOptionPane.

12. Limpar os campos somente após uma operação considerada válida.

Verificação: A tela cria objetos válidos e mantém os dados quando a correção do usuário é necessária.

Etapa 4 – Revisar responsabilidades

13. Extrair painéis ou formulários quando a classe principal concentrar construção demais.

14. Remover regras de negócio escritas diretamente em componentes visuais.

15. Documentar o fluxo entre inicialização, view e objeto de domínio.

16. Executar os testes obrigatórios e identificar as evidências para o relatório.

Verificação: A interface permanece simples e as responsabilidades estão separadas de forma justificável.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Adicionar atalhos de teclado e ordem de foco coerente.
- Criar um tema visual discreto sem comprometer legibilidade.
- Adicionar validação reutilizável para ISBN ou outro identificador do domínio.

Evidências para o relatório

- Diagrama simples das classes da tela inicial e suas responsabilidades.
- Código da janela, dos painéis e dos objetos de domínio usados no cadastro.
- Capturas da tela em estado inicial, preenchido e com validação de erro.
- Registro dos testes manuais e das decisões de composição adotadas.

Objetivo

Evoluir a interface para múltiplas telas, menus e listagem de livros, escolhendo layouts adequados e um modelo de tabela próprio.

Etapa 1 – Comparar layouts

1. Criar pequenos painéis demonstrativos com BorderLayout e GridLayout.
2. Redimensionar as janelas e registrar como cada layout distribui os componentes.
3. Criar uma versão controlada com layout nulo apenas para demonstrar suas limitações.
4. Escolher e justificar a combinação de layouts para a aplicação final.

Verificação: A escolha de layout está vinculada ao comportamento observado, não apenas à aparência inicial.

Etapa 2 – Criar navegação e menus

5. Criar cartões para início, cadastro e consulta com CardLayout.
6. Adicionar barra de menus com opções de navegação, ação e saída.
7. Definir atalhos e confirmar que itens equivalentes produzem o mesmo comando.
8. Centralizar a troca de cartões em um método ou controlador de navegação.

Verificação: Menu e navegação conduzem aos cartões corretos sem duplicar lógica.

Etapa 3 – Implementar a tabela

9. Criar LivroTableModel a partir de AbstractTableModel.

10. Definir nomes, tipos e editabilidade das colunas de forma explícita.
11. Carregar uma coleção em memória e notificar o modelo após alterações.
12. Configurar seleção, larguras e apresentação de valores sem misturar persistência na view.

Verificação: A tabela representa o modelo e reage corretamente a inclusões, alterações e remoções.

Etapa 4 – Integrar e validar

13. Conectar cadastro e consulta por uma fonte de dados em memória.
14. Editar o item selecionado e atualizar somente após validação.
15. Tratar tabela vazia, filtro sem resultado e seleção ausente.
16. Registrar capturas e resultados em diferentes tamanhos de janela.

Verificação: O fluxo entre cadastro, menu e tabela é reproduzível e visualmente estável.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Adicionar filtro por título ou categoria usando `TableRowSorter`.
- Criar renderizador para destacar livros indisponíveis.
- Comparar duas composições de layout e justificar a escolhida.

Evidências para o relatório

- Mapa das telas e menus implementados.
- Código dos painéis, gerenciadores de layout e navegação com CardLayout.
- Implementação de TableModel para livros, sem copiar dados para células manualmente.
- Capturas em dois tamanhos de janela e testes de navegação e tabela.

Objetivo

Controlar ações do usuário e ciclo de vida da janela sem bloquear a interface nem duplicar regras de negócio.

Etapa 1 – Mapear eventos

1. Listar botões, menus, campos e janelas que produzem eventos relevantes.
2. Definir para cada fonte o listener, a ação esperada e o estado afetado.
3. Registrar listeners em um ponto previsível da construção da tela.
4. Evitar criar múltiplos listeners para a mesma responsabilidade sem necessidade.

Verificação: Cada evento possui fonte, listener e consequência identificáveis.

Etapa 2 – Implementar ações

5. Criar comandos para salvar, pesquisar, limpar e sair.
6. Reutilizar o mesmo comando entre botão e item de menu quando o comportamento for igual.
7. Validar o estado antes de executar alterações.
8. Capturar falhas na fronteira da interface e converter em mensagem útil.

Verificação: As ações são consistentes e não repetem regras em cada componente.

Etapa 3 – Controlar a janela

9. Configurar fechamento controlado em vez de encerrar imediatamente.

10. Usar WindowAdapter para solicitar confirmação quando houver dados não salvos.
11. Testar confirmação, cancelamento e fechamento sem alterações pendentes.
12. Liberar recursos temporários quando a janela for encerrada.

Verificação: O ciclo de vida da janela preserva o estado e respeita a decisão do usuário.

Etapas 4 – Manter a interface responsiva

13. Simular uma operação demorada acionada pela interface.
14. Executá-la fora da Event Dispatch Thread usando mecanismo apropriado, como SwingWorker.
15. Desabilitar a ação durante a execução e restaurá-la em caso de sucesso ou falha.
16. Atualizar a interface na thread correta e registrar o resultado observado.

Verificação: A operação não congela a interface e não pode ser disparada em duplicidade.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Criar uma classe Action compartilhada por botão, menu e atalho.
- Adicionar cancelamento cooperativo a uma tarefa simulada.
- Criar teste de controlador sem instanciar a janela completa.

Evidências para o relatório

- Tabela relacionando fonte, evento, listener e efeito no sistema.
- Código dos controladores e listeners, incluindo fechamento confirmado.
- Demonstração de operação demorada fora da Event Dispatch Thread.
- Testes de clique repetido, cancelamento, falha e recuperação da interface.

Objetivo

Persistir livros e categorias em MySQL com JDBC, usando consultas parametrizadas, transações e fechamento determinístico de recursos.

Etapa 1 – Preparar banco e driver

1. Criar banco e tabelas de livros, autores e categorias com chaves e restrições coerentes.
2. Adicionar o Connector/J pela ferramenta de dependências adotada.
3. Manter URL, usuário e senha fora do código-fonte e registrar as variáveis necessárias.
4. Criar teste de conexão que não revele senha em logs ou mensagens.

Verificação: O esquema é reproduzível e a conexão usa configuração externa.

Etapa 2 – Ler informações

5. Criar consulta parametrizada por título ou categoria.
6. Usar try-with-resources para conexão, statement e result set.
7. Mapear cada linha para um objeto ou DTO sem depender da interface Swing.
8. Exibir o resultado na tabela da atividade anterior.

Verificação: A listagem vem do banco e todos os recursos são fechados deterministicamente.

Etapa 3 – Alterar informações

9. Implementar inserção e recuperar o identificador gerado quando necessário.
10. Implementar atualização e exclusão com parâmetros.

11. Verificar a contagem de linhas afetadas antes de informar sucesso.
12. Converter SQLException em falha da aplicação sem expor detalhes sensíveis na tela.

Verificação: CRUD usa parâmetros e informa corretamente sucesso, ausência ou falha.

Etapa 4 - Aplicar transação

13. Criar uma operação que altere duas tabelas como uma única unidade lógica.
14. Desabilitar auto-commit, confirmar apenas após todos os comandos e desfazer em caso de falha.
15. Restaurar o estado esperado e fechar a conexão mesmo quando ocorrer exceção.
16. Executar um teste forçando falha no segundo comando.

Verificação: O teste comprova atomicidade: ou todas as alterações persistem, ou nenhuma persiste.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Usar DataSource e comparar com conexão direta via DriverManager.
- Adicionar paginação à consulta da tabela.
- Criar teste de integração com esquema isolado.

Evidências para o relatório

- Script de criação do esquema e dados mínimos de teste.
- Código de conexão, consultas parametrizadas e mapeamento de resultados.
- Log ou capturas das operações criar, ler, alterar e excluir.
- Teste de rollback e descrição de como as credenciais são fornecidas sem serem incluídas no repositório.

 	Disciplina: Prática de Programação Orientada a Objetos Título da aula: DTO/VO, MVC e Data Access Object aplicados à solução	ROTEIRO 5
---	---	------------------

Objetivo

Reorganizar a aplicação em camadas, separar apresentação, controle e persistência e tornar a troca da implementação de dados verificável.

Etapa 1 – Definir contratos

1. Identificar dados de tela, objetos de domínio e valores com regras próprias.
2. Criar DTOs de entrada e saída sem comportamento de persistência.
3. Criar ao menos um VO, como Isbn, com validação centralizada.
4. Documentar conversões entre DTO, domínio e linhas do banco.

Verificação: Cada tipo possui finalidade clara e conversões explícitas.

Etapa 2 – Separar MVC

5. Manter a view responsável por apresentar dados e emitir intenções do usuário.
6. Criar controlador para coordenar validação, serviço e atualização da view.
7. Concentrar regras do caso de uso em serviço ou modelo, não em listeners.
8. Usar TableModel apenas como adaptação entre lista de objetos e JTable.

Verificação: A interface pode mudar sem transferir regras de negócio para componentes Swing.

Etapa 3 – Introduzir DAO

9. Definir LivroDAO com operações necessárias ao caso de uso.

10. Mover SQL e mapeamento JDBC para JdbcLivroDAO.
11. Criar MemoriaLivroDAO para testes e demonstrações.
12. Injetar a interface DAO pelo construtor do serviço ou controlador.

Verificação: A troca de implementação ocorre na composição da aplicação, sem alterar a view.

Etapa 4 – Testar as fronteiras

13. Testar serviço e controlador com DAO em memória.
14. Simular falha no DAO e verificar a resposta do controlador.
15. Executar o mesmo contrato básico contra a implementação JDBC.
16. Revisar dependências e remover imports de JDBC das camadas superiores.

Verificação: Os testes demonstram desacoplamento, mensagens previsíveis e equivalência do contrato.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Criar fábrica ou composição central para selecionar o DAO por configuração.
- Adicionar caso de uso de empréstimo envolvendo mais de um DAO.
- Criar teste arquitetural simples para dependências entre pacotes.

Evidências para o relatório

- Diagrama de pacotes ou classes com dependências entre view, controller, service e DAO.
- DTOs/VOs e mapeamentos explícitos usados na aplicação.
- Interface DAO e implementações em memória e JDBC.
- Testes do controlador ou serviço com DAO substituto, sem banco real.

Objetivo

Mapear e persistir o domínio da biblioteca com Hibernate, delimitando sessão, transação e carregamento de associações.

Etapa 1 – Configurar persistência

1. Adicionar dependências Hibernate e driver compatíveis com o ambiente homologado.
2. Configurar acesso ao banco fora do código-fonte sensível.
3. Criar uma SessionFactory, na API nativa do Hibernate, ou uma EntityManagerFactory, na Jakarta Persistence, uma única vez durante a inicialização.
4. Registrar estratégia de criação ou validação do esquema usada apenas para o ambiente didático.

Verificação: A infraestrutura inicia uma vez, conecta e encerra seus recursos de forma controlada.

Etapa 2 – Mapear entidades

5. Mapear Livro, Autor, Categoria e ao menos um relacionamento.
6. Definir identificadores, nulabilidade e tamanhos coerentes com o esquema.
7. Escolher acesso por campos ou propriedades e manter consistência.
8. Evitar incluir coleções preguiçosas em toString, igualdade ou logs inadvertidamente.

Verificação: Os mapeamentos representam o domínio e o esquema sem recursão ou estado inconsistente.

Etapa 3 – Persistir com transações

9. Criar implementação Hibernate do contrato de repositório ou DAO.
10. Abrir sessão, iniciar transação, executar a operação e confirmar apenas em caso de sucesso.
11. Executar rollback e fechamento em caso de falha.
12. Integrar cadastro e consulta à interface sem expor a sessão à view.

Verificação: O fluxo desktop persiste e consulta por uma fronteira de dados estável.

Etapa 4 – Observar consultas

13. Habilitar SQL para o ambiente de desenvolvimento sem registrar credenciais.
14. Executar uma consulta que carregue relacionamento usado pela tela.
15. Identificar consultas repetidas e ajustar fetch ou query de forma pontual.
16. Comparar a implementação Hibernate à JDBC quanto a código, SQL e controle explícito.

Verificação: A decisão de carregamento é demonstrada pelo SQL e não depende de acesso fora da sessão.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Adicionar paginação à consulta Hibernate.

- Mapear concorrência otimista com atributo de versão.
- Executar o mesmo teste de contrato nas implementações JDBC e Hibernate.

Evidências para o relatório

- Entidades mapeadas e configuração de persistência utilizada.
- Código de sessão/transação e implementação Hibernate do repositório.
- SQL gerado para inclusão, consulta e relacionamento selecionado.
- Testes de persistência, rollback e carregamento de associações.

 	Disciplina: Prática de Programação Orientada a Objetos Título da aula: Módulo web com HTML, Servlets, JSP e JSTL	ROTEIRO 7
---	--	------------------

Objetivo

Disponibilizar consulta web do acervo, usando Servlet como controlador, JSP como view e JSTL para apresentação sem lógica de negócio na página.

Etapa 1 – Preparar o módulo web

1. Criar módulo web compatível com o servidor homologado.
2. Adicionar dependências de Servlet, JSP e JSTL com escopos adequados.
3. Configurar o servidor e publicar uma página HTML estática de verificação.
4. Registrar instruções de execução e o contexto da aplicação.

Verificação: O servidor publica o módulo e a página de verificação responde.

Etapa 2 – Criar controlador

5. Criar Servlet para receber filtro de acervo.
6. Validar e normalizar parâmetros antes de chamar o serviço.
7. Colocar DTOs de resultado no escopo da requisição.
8. Encaminhar para uma JSP dentro de WEB-INF.

Verificação: O controlador prepara os dados e a JSP não é usada como ponto de entrada direto.

Etapa 3 – Construir JSP e JSTL

9. Criar HTML semântico com título, formulário e tabela de resultados.

10. Declarar a taglib correta para a versão adotada.
11. Usar `c:if`, `c:choose`, `c:forEach` e saída escapada conforme a necessidade.
12. Evitar scriptlets e chamadas diretas ao DAO dentro da página.

Verificação: A view exibe dados, vazio e mensagens com JSTL/EL, sem Java embutido.

Etapa 4 – Tratar falhas

13. Criar um cenário controlado de falha no serviço.
14. Converter a falha em mensagem apropriada na fronteira web.
15. Demonstrar `c:catch` em um exemplo didático restrito e discutir por que não substitui tratamento centralizado.
16. Testar caracteres especiais e confirmar que a saída não injeta marcação não desejada.

Verificação: Falhas são tratadas sem detalhes internos e a saída permanece segura e legível.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Adicionar paginação e preservação dos parâmetros do filtro.
- Criar uma página de detalhes com rota e validação de identificador.
- Adicionar filtro de codificação e cabeçalhos básicos de segurança.

Evidências para o relatório

- Estrutura do módulo web, dependências e configuração do servidor.
- Servlet/controlador, JSPs, taglibs e HTML produzidos.
- Capturas de lista, filtro, resultado vazio, erro controlado e do aprimoramento escolhido.
- Teste demonstrando reutilização do serviço/repositório criado anteriormente.

Objetivo

Gerar uma planilha de acervo e empréstimos a partir dos serviços existentes, com dados corretos, formatação reutilizável e arquivo validado.

Etapa 1 – Definir o relatório

1. Escolher relatório de acervo, empréstimos ou disponibilidade e definir colunas e filtros.
2. Criar DTO específico para cada linha do relatório.
3. Obter os dados pelo serviço existente, sem consultar banco diretamente na view.
4. Definir formato XLSX e registrar a dependência Apache POI usada.

Verificação: Colunas, tipos e fonte dos dados estão definidos antes da formatação.

Etapa 2 – Criar a planilha

5. Criar um workbook e uma planilha com nome válido.
6. Criar cabeçalho e estilos reutilizáveis para texto, data e número.
7. Preencher linhas mantendo os tipos das células.
8. Adicionar congelamento de cabeçalho e filtro automático quando úteis.

Verificação: A planilha contém dados tipados e estilos reutilizados de forma consistente.

Etapa 3 – Integrar à aplicação

9. Adicionar ação de exportar no menu da aplicação Swing.
10. Usar JFileChooser com extensão sugerida e confirmação de sobrescrita.

11. Executar geração potencialmente demorada fora da Event Dispatch Thread.
12. Informar sucesso apenas após fechamento completo do arquivo.

Verificação: A exportação respeita a escolha do usuário e não bloqueia a interface.

Etapas 4 - Validar o arquivo

13. Reabrir o XLSX com Apache POI e conferir planilha, cabeçalhos e linhas.
14. Comparar totais e quantidade de registros com a consulta de origem.
15. Testar conjunto vazio, textos longos, acentos e caracteres especiais.
16. Abrir o arquivo em aplicativo compatível e registrar evidências visuais.

Verificação: O arquivo é tecnicamente legível e seus dados correspondem à origem.

Checklist de entrega

- O projeto compila e executa a partir das instruções registradas.
- Os testes obrigatórios foram executados e as evidências estão identificadas.
- O código não contém segredos, credenciais ou dados sensíveis.
- O relatório explica decisões e resultados, sem apenas repetir o código.

Extensões para criatividade e aprofundamento

- Escolher uma ou mais extensões, mantendo a solução principal correta:
- Adicionar planilha de resumo com fórmulas ou contagens por categoria.
- Aplicar formatação condicional a itens atrasados.
- Criar teste automatizado que valide células sem depender do Excel instalado.

Evidências para o relatório

- Código do serviço de relatório e integração com a interface Swing.

- Planilha XLSX com cabeçalho, dados e formatos, acompanhada da evidência do aprimoramento escolhido.
- Capturas do arquivo aberto no Excel ou aplicativo compatível.
- Testes de arquivo vazio, caracteres especiais, totais e quantidade de linhas.